



Final Group Project

Cloud Security Hardening Plan

05.03.2025

Marc Andreu Grillo Aguilar - aguilm
Victoria Ifedolapo Nathaniel - vnath447
Nimal Kurien Thomas - nthoma14
ENPM665-0201

Executive Summary.....	5
Background: Connection to Midterm.....	6
Public Exposure of Critical Services.....	6
Absence of Subnet Segmentation.....	7
Flat Security Group Architecture.....	7
Inadequate Identity Controls.....	7
Lack of Traffic Monitoring and Guardrails.....	7
Limited Scalability and Fault Tolerance.....	8
Misconfigured Application Deployment.....	8
Objectives.....	8
Infrastructure and Compute Security.....	8
Objective 1: Deploying Resources into Private Subnets.....	8
Goal.....	8
Key Improvement.....	9
Justification.....	9
Objective 2: Enforcing IAM Least Privilege for Instance Roles.....	9
Goal.....	9
Key Improvement.....	9
Justification.....	9
Network and Access Control Security.....	9
Objective 1: Enforce Private Network Architecture and Minimize Public Exposure.....	9
Goal.....	9
Key Improvement 1.....	9
Justification.....	10
Key Improvement 2.....	10
Justification.....	10
Objective 2: Harden Security Groups and Access Control Boundaries.....	10
Goal.....	10
Key Improvement 1.....	10
Justification.....	10
Disaster Recovery and Monitoring (this was an extra one we did).....	11
Goal.....	11
Key Improvement.....	11
Justification.....	11
Implementation Plan.....	11
Infrastructure and Compute Security.....	11
Deploying Resources into Private Subnets.....	11
AWS Services Used.....	11
Technical Steps.....	11

Enforcing IAM Least Privilege for Instance Roles.....	12
AWS Services Used.....	12
Technical Steps.....	12
Network and Access Control Security.....	14
Enforce Private Network Architecture and Restrict Public IP Access.....	14
AWS Services Used.....	14
Technical Steps.....	14
Replace Public EC2 Exposure with Application Load Balancer (ALB).....	15
AWS Services Used.....	15
Technical Steps.....	15
Harden Security Groups and Access Control Boundaries.....	15
AWS Services Used.....	15
Technical Steps.....	15
Disaster Recovery and Monitoring (this was an extra one we did).....	16
Implement VPC Flow Logs and GuardDuty for Traffic Visibility and Threat Detection...	16
AWS Services Used.....	16
Technical Steps.....	16
Validation and Testing.....	16
Deploying the EC2 Instances into Private Subnets.....	16
Validation.....	16
Replace Public EC2 Exposure with Application Load Balancer (ALB).....	16
Validation.....	16
Ingress/Egress Rules Validation for Security Groups.....	17
Testing Strategy.....	17
Manual Penetration Attempt.....	17
AWS Console Review.....	17
Results.....	17
Public IP Access Restriction Validation.....	17
Testing Strategy.....	17
VPC Reachability Analyzer.....	17
CURL/Wget Tests.....	18
Results.....	18
Supporting Evidence.....	18
ALB Health Check and Access Verification.....	18
Testing Strategy.....	18
ALB Console Monitoring.....	18
Load Simulation.....	18
Results.....	18

Supporting Evidence: Image 10: ALB Target Group Health Checks.....	19
Impact Assessment.....	19
Before and After Security Posture Comparison.....	19
Technical Control Improvements.....	20
Isolation via Private Subnets.....	20
Restricted Ingress Access.....	20
Monitoring and Forensics Readiness.....	20
Defense-in-Depth.....	20
Evidence of Control Effectiveness.....	20
Blocked Unauthorized Access.....	20
Health Checks.....	20
Residual Risks and Limitations.....	21
Performance and Cost Trade-Offs.....	21
Lessons Learned and Group Reflection.....	22
Challenges Faced.....	22
Lessons Learned.....	22
Future Improvements.....	23
Appendix.....	24
Midterm vulnerability fix 1 (Image 1).....	24
The actual EC2 in the private subnet (Image 2).....	25
Bastion Host Implementation (Image 3).....	25
Application Load Balancer and Segmentation (Images 4, 5).....	26
Network Monitoring Enabled (Image 6).....	27
Subnet Isolation and Design Confirmation (Images 7, 8).....	27
EC2 IAM Role Enforcement, Post-Hardening (Image 9).....	28
ALB Health Check Validated (Images 10, 11, 12, 13, 14).....	29
Finding 1: Abuse of Unrestricted Instance Metadata Service (IMDSv1).....	31
Remediation Implemented.....	31
Proof: Image 1.....	31
Finding 2: Privilege Escalation via Over-Permissive IAM Policies.....	31
Remediation Implemented.....	31
Proof: Image 2.....	31
Finding 3: Hardcoded Root Password.....	31
Remediation Implemented.....	31
Finding 4: Directory Permissions (755).....	32
Remediation Implemented.....	32
Finding 5: Lack of Encryption in Storage.....	33
Remediation Implemented.....	33
Finding 6: Outdated crypto-js.....	33

Remediation Implemented.....	33
Finding 7: XXE in libxmljs2.....	33
Remediation Implemented.....	33
Finding 9: SSH (Port 22) Open.....	33
Remediation Implemented.....	33
Finding 10: Kernel & Jinja2 CVEs.....	33
Remediation Implemented.....	33
Finding 11: No Patch Management.....	34
Remediation Implemented.....	34
Finding 12: Juice Shop + MariaDB on Same EC2.....	34
Remediation Implemented.....	34
Finding 13: Orphaned Private Subnet.....	34
Remediation Implemented.....	34
Finding 14: Lack of Monitoring.....	34
Remediation Implemented.....	34
Proof: Image 6.....	34
Finding 15: Juice Shop on Port 3000 Publicly Exposed.....	34
Remediation Implemented.....	34
Finding 16: No IAM Role on Instance.....	34
Remediation Implemented.....	34
Proof: Image 2.....	34
Finding 17: Unrestricted Egress.....	34
Remediation Implemented.....	34
Proof Image 5.....	35
Finding 18: No AZ Redundancy.....	35
Remediation Implemented.....	35
Proof: Image 7 Subnet Map Screenshot.....	35
Finding 19: No Backups.....	35
Remediation Implemented.....	35
Finding 20: No Health Checks.....	35
Remediation Implemented.....	35
Proof: Image 10.....	35

Executive Summary

This Cloud Security Hardening Plan provides an updated template for the Juice Shop e-commerce platform, which is hosted in Amazon Web Services (AWS). The primary goal of this assessment is continuing on the findings found in the previous report in order to design and implement solutions for confidentiality, integrity, availability, and resilience of the Juice Shop platform.

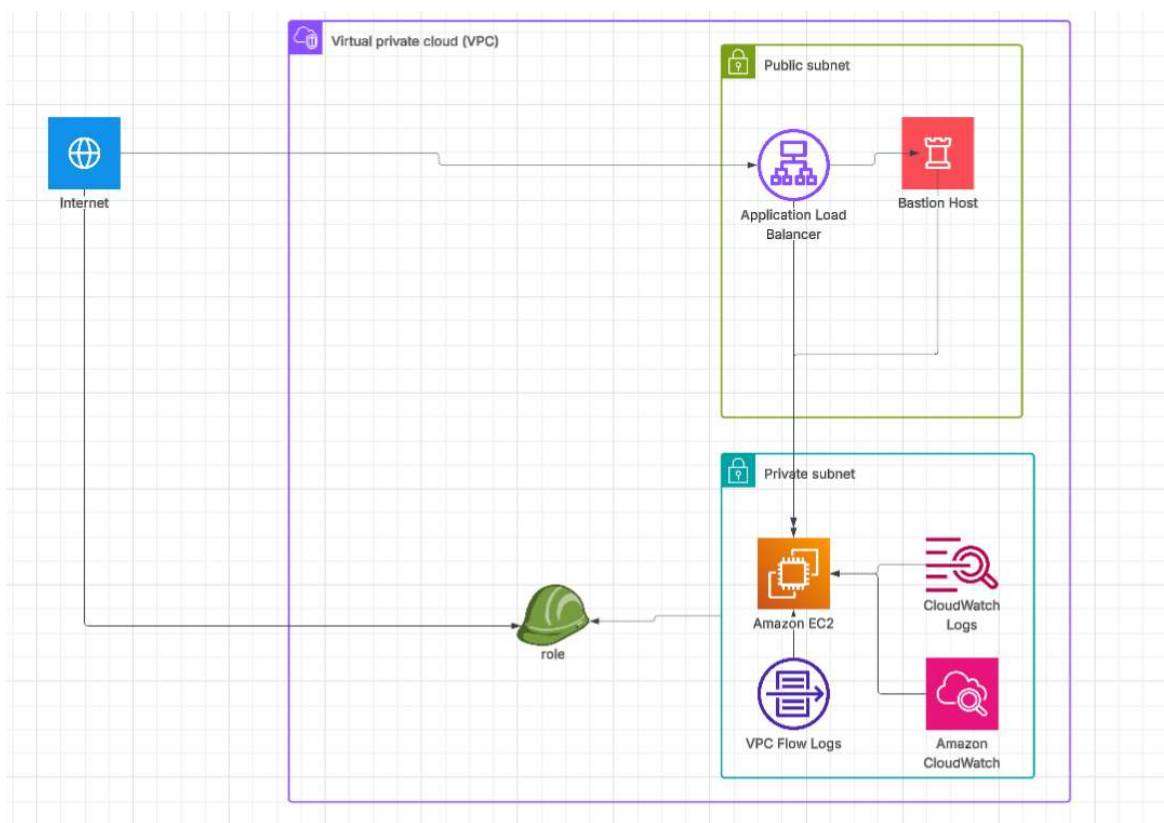
This plan is focused on two areas: Infrastructure and Compute Security and Network and Access Control Security.

Measures for Infrastructure and Compute Security include:

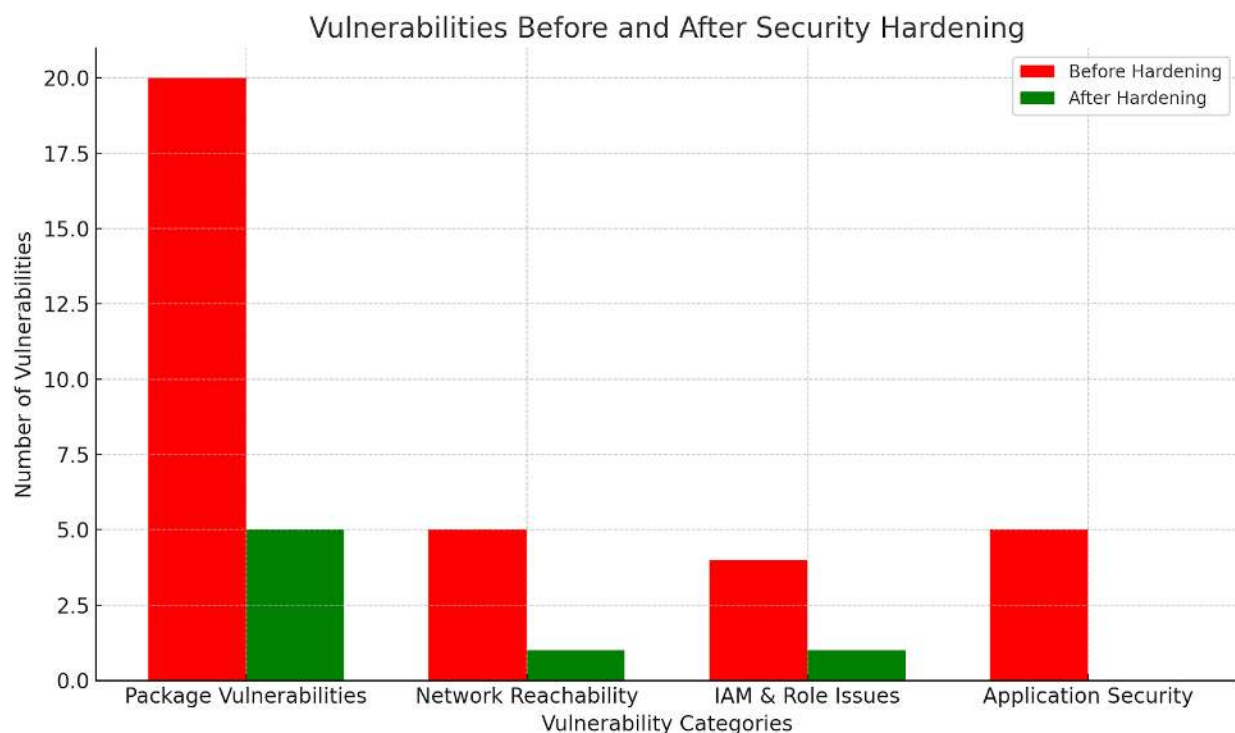
- Patching the EC2 instance
- Deploying resources into private subnets

Whilst the measures for Network and Access Control Security involve:

- Implementing private VPC architectures
- Restricting public IP access



Here's our VPC architecture



These improvements were implemented based on remediations consistent with the AWS Well-Architected Framework, NIST, and the CIS AWS Foundations Benchmark v4.0.1. Using a combination of AWS-native tools including Amazon Inspector, AWS Systems Manager and the CIS Amazon Web Services Foundations Benchmark as a third-party benchmark, the security team improved existing network configurations.

Background: Connection to Midterm

During the midterm project, our group conducted a vulnerability assessment on the initial deployment of the OWASP Juice Shop application hosted in a default Amazon EC2 instance. The infrastructure setup was defined by a baseline CloudFormation template that exposed a variety of fundamental security flaws across the computing, networking, and access control planes.

This section revisits the exact weaknesses discovered in that infrastructure and justifies why our selected focus area—Network and Access Control Security—was the most critical domain requiring immediate remediation.

Public Exposure of Critical Services

The midterm CloudFormation template provisioned an EC2 instance into a public subnet with an associated public IP address. Furthermore, all incoming traffic on ports 22 (SSH), 80



(HTTP), and 443 (HTTPS) was allowed from 0.0.0.0/0, violating the principle of least privilege. This open access created multiple entry points for attackers, particularly when weak credentials or misconfigured services are present.

Absence of Subnet Segmentation

No distinction was made between public and private subnets. All resources were provisioned into a single publicly routable subnet. This meant that internal components like the MariaDB database, application logic, and EC2 metadata could be directly targeted by external actors, increasing the blast radius of a potential breach. Best practices recommend placing internal compute workloads behind NAT gateways in private subnets, which was not followed.

Flat Security Group Architecture

The midterm template used a single security group that allowed broad TCP access, again from 0.0.0.0/0. It lacked directional traffic filtering and had no segmentation between application layer access (port 3000) and administrative access (SSH). Without separating the concerns of user access versus application logic, any compromise could lead to lateral movement within the environment.

Inadequate Identity Controls

The EC2 instance did not use IAM roles for secure identity and access management. Instead, any AWS SDK or CLI invocation from the instance would require manually configured long-term access keys, increasing the risk of key leakage. There was no boundary or scope limitation in place to constrain access if the instance or any applications running on it were compromised.

Lack of Traffic Monitoring and Guardrails

No network monitoring, logging, or automated guardrails were present in the architecture. This prevented the team from detecting brute-force attacks, scanning attempts, or outbound exfiltration behavior during simulated tests. Services like AWS VPC Flow Logs, GuardDuty, or AWS Network Firewall could have provided visibility into these suspicious behaviors.

Limited Scalability and Fault Tolerance

The midterm environment used a single EC2 instance without an Auto Scaling Group (ASG) or Elastic Load Balancer (ALB). This single point of failure not only risked availability but also concentrated the attack surface on a single IP endpoint. Moreover, the application had no ability to scale during simulated load tests.

Misconfigured Application Deployment

Lastly, the Juice Shop application was hosted directly on port 3000 without port forwarding, NGINX proxying, or integration with industry-standard platforms like Amazon ECS or EKS. This non-standard port required manually updating the security group to enable external access, a high-friction and error-prone process.

Objectives

Based on the above findings, we determined that the most impactful remediation strategy would center on "Infrastructure and Compute Security" as well as "Network and Access Control Security."

Fixing public exposure, redesigning subnet layout, locking down security groups, and introducing ALB and ASG would fundamentally change the security posture and operational resilience of the deployment.

By focusing on network hardening, we aimed to implement layered security controls and adopt an AWS Well-Architected approach to improve confidentiality, integrity, and availability. This would address the root issues identified in the midterm project and provide a secure foundation for future enhancements, such as zero-trust models and microsegmentation.

Our improved infrastructure directly builds upon these findings and is supported by evidence in the implementation and validation sections that follow.

Infrastructure and Compute Security

Objective 1: Deploying Resources into Private Subnets

Goal

We need to ensure that the EC2 Instance uses the PrivateSubnet, rather than the PublicSubnet that it was set to in the initial template. The previous template had the correct definitions created to allow the private subnet to be used, it just needed to be set within the configuration for the EC2 instance.

Key Improvement

With this change, the EC2 instance will not be able to reach the outside internet, since the private subnet does not have a route to an internet gateway defined in the template.

Justification

In the previous report, it was noted that there existed a public and private subnet. However, the private subnet was not in active use. The private subnet was not configured properly to allow resources within it, such as databases, to access the internet. The private subnet also lacked any Network ACLs to enforce traffic between the subnets, such as explicitly denying cross-subnet traffic.

Objective 2: Enforcing IAM Least Privilege for Instance Roles

Goal

For this improvement, we will be creating a new IAM role and attaching it to the EC2 instance.

Key Improvement

With this IAM role added to the template, the EC2 instance will have its own proper IAM role. The role grants minimal and scoped-down permissions to read objects and aligns with best practices for least privilege access. This also helps to separate the role trust from the permissions logic and will be easier to identify for managing future policy changes.

Justification

The previous iteration of the CloudFormation template did not contain an IAM role for the EC2 instance.

Network and Access Control Security

Objective 1: Enforce Private Network Architecture and Minimize Public Exposure

Goal

With the EC2 instances, holding sensitive workloads, in the private subnets as described previously, public internet. We will limit public access to what is necessary, via protected and monitored services.

Key Improvement 1

To communicate with the EC2 instance, deployed in the private subnet, from the public subnet, we will access them via a secure bastion host in the public subnet. This bastion host is protected with a security group that only allows SSH access from trusted IP



addresses. This provides secure administrative access to the migration of the instance via SSH tunneling.

Justification

The midterm revealed unrestricted SSH access on the main EC2 instance, which significantly increased the attack surface. Placing the workload in a private subnet replaces direct access with secure access instead.

Key Improvement 2

Introduce an Application Load Balancer (ALB) that sits in the public subnet. It forwards HTTP traffic to the EC2 instance in the private subnet, where the Juice Shop application is running. The ALB will send traffic requests to a target group where the health check validates the EC2 instance at the rootpath `"/`.

Justification

Previously, the EC2 instance accepted traffic directly. Having an ALB separates the internet traffic from internal resources. It also increases scalability, a centralized inspection point, and allows for future enhancements HTTPS/TLS Enforcement and WAF.

Objective 2: Harden Security Groups and Access Control Boundaries

Goal

Refine network-level access controls by adopting the AWS best practices of least privilege and implementing layered defenses through AWS Security Groups and Network ACLs.

Key Improvement 1

We restructured the security groups to strictly restrict traffic. The security group for the ALB allows inbound TCP traffic on port 80 from 0.0.0.0/0. The security group for the EC2 Instance allows for inbound traffic on port 3000, but only from the Security Group for the ALB. This means no outside entity can reach the EC2 Instance directly, even in the VPC.

Justification

In the previous setup, the EC2 instance exposed global inbound access on multiple ports. This could lead to additional risks of brute-force attacks and lateral movement access to other systems.

This new configuration ensures that only the ALB can talk to the instance on the app layer. Therefore, if the backend system is improperly allowed or available at the internet layer, it is still safe from direct exposure.

Disaster Recovery and Monitoring (this was an extra one we did)

Goal

Enable visibility into the traffic.

Key Improvement

We enabled VPC Flow Logs to receive metadata about IP levels across all subnets. This allows better visibility of network activity. Logs are sent to CloudWatch so that anomalous traffic patterns can be detected. We also set up Network ACLs (NACLs) that can limit traffic across subnets and enforce rules to block traffic in one direction (at the packet level).

Justification

The midterm deployment lacked traffic monitoring, and by extension, all scanning attempts, brute-force logins, and egress traffic. All of these were able to go undetected.

In contrast, with flow logs and controlled ACLs, all traffic is viewable, observable, and controlled, which is compliant with the requirements of defensive-in-depth principles and auditing.

Implementation Plan

Infrastructure and Compute Security

Deploying Resources into Private Subnets

AWS Services Used

EC2, VPC, Subnets, NAT Gateway, Route Tables, CloudFormation

Technical Steps

1. Modify the template to include both public and private subnets within the VPC

Unset

```
PublicSubnet1: # The actual template has more properties listed for this
```

```
  CidrBlock: 10.0.1.0/24
```

```
  MapPublicIpOnLaunch: true
```

```
PrivateSubnet1: # The actual template has more properties listed for this
```

```
  CidrBlock: 10.0.3.0/24
```

```
  MapPublicIpOnLaunch: false
```

2. With the AutoScalingGroup, set the VPCZoneIdentifiers to the private subnets

Unset

```
AutoScalingGroup: # This gets attached to the EC2 LaunchTemplate defined
  Type: AWS::AutoScaling::AutoScalingGroup
  Properties:
    VPCZoneIdentifier:
      - !Ref PrivateSubnet1
      - !Ref PrivateSubnet2
    ...more configurations
```

Enforcing IAM Least Privilege for Instance Roles

AWS Services Used

EC2, IAM

Technical Steps

To accomplish this, we'll need to make four changes to the template:

1. Create a ManagedPolicy, with read-only privileges for S3 buckets
2. Create a IAM role, referencing the ManagedPolicy
3. Create a InstanceProfile to be able to attach the IAM role to the EC2 instance
4. Reference the InstanceProfile in the EC2 LaunchTemplate configuration

Here's the WebServerManagedPolicy that we'll need:

Unset

```
WebServerManagedPolicy:
  Type: "AWS::IAM::ManagedPolicy"
  Properties:
    ManagedPolicyName: "EC2ReadOnlyS3Policy"
    Description: "Grants read-only access to S3 for EC2"
    PolicyDocument:
      Version: "2012-10-17"
      Statement:
        - Effect: Allow
          Action:
```

```
- "s3:GetObject"  
Resource: "*"
```

Next is the `WebServerRole`, with the reference to the above policy:

```
Unset  
WebServerRole:  
  Type: "AWS::IAM::Role"  
  Properties:  
    AssumeRolePolicyDocument:  
      Version: "2012-10-17"  
      Statement:  
        - Effect: Allow  
          Principal:  
            Service: ec2.amazonaws.com  
          Action: sts:AssumeRole  
    ManagedPolicyArns:  
      - !Ref WebServerManagedPolicy
```

Here's the `InstanceProfile` that refers to the above role:

```
Unset  
WebServerInstanceProfile:  
  Type: "AWS::IAM::InstanceProfile"  
  Properties:  
    Roles:  
      - !Ref WebServerRole
```

And finally the reference to the `InstanceProfile` in the EC2 configuration

```
Unset

LaunchTemplate:
  Type: AWS::EC2::LaunchTemplate
  Properties:
    LaunchTemplateName: JuiceShopLT
    LaunchTemplateData:
      InstanceType: t2.micro
      ImageId: !Ref AMIID
      KeyName: !Ref KeyPairName
      IamInstanceProfile:
        Arn: !GetAtt WebServerInstanceProfile.Arn
    ... More Configuration below
```

Network and Access Control Security

Enforce Private Network Architecture and Restrict Public IP Access

AWS Services Used

EC2, VPC, Subnets, Security Groups, NAT Gateway, Route Tables

Technical Steps

We need to isolate the backend infrastructure components from direct internet exposure while ensuring availability and controlled access.

1. Remove `MapPublicIpOnLaunch: true` from the private subnet declaration.
2. Add a NAT Gateway to the public subnet
 - a. The NAT Gateway needs to be provisioned in the public subnet and attached to the route table for the private subnet to allow secure outbound internet access for patching and updates.
3. Deploy the updated stack using AWS CloudFormation

Here's the updated information for the cloud template:

```
Unset

PublicSubnet1:
  CidrBlock: 10.0.1.0/24
  MapPublicIpOnLaunch: true
```

```
PrivateSubnet1:
  CidrBlock: 10.0.3.0/24
  MapPublicIpOnLaunch: false
```

Replace Public EC2 Exposure with Application Load Balancer (ALB)

AWS Services Used

ALB, Target Groups, EC2, CloudFormation

Technical Steps

1. Create an ALB resource in the CloudFormation template across two public subnets
2. Configure a HTTP listener on port 80
 - a. It needs a forwarding action to a target group containing the EC2 instances
3. Define a target group with health checks enabled on the / path
4. Output the DNS of the ALB as the new public access point for Juice Shop

Here's the configuration needed for the template

```
Unset
ALB:
  Type: AWS::ElasticLoadBalancingV2::LoadBalancer
  Properties:
    Subnets:
      - !Ref PublicSubnet1
      - !Ref PublicSubnet2
    SecurityGroups:
      - !Ref ALBSecurityGroup
```

Harden Security Groups and Access Control Boundaries

AWS Services Used

EC2, Security Groups, VPC, CloudFormation

Technical Steps

1. Define three separate security groups:

Unset

ALBSG: # Allows HTTP from 0.0.0.0/0

InstanceSG: # Allows port 3000 only from ALBSG

BastionSG: # Allows SSH only from known IPs

2. Associate the EC2 instance with InstanceSG.
3. Deploy a bastion host in the public subnet using BastionSG for SSH access.

Disaster Recovery and Monitoring (this was an extra one we did)

Implement VPC Flow Logs and GuardDuty for Traffic Visibility and Threat Detection

AWS Services Used

VPC Flow Logs, CloudWatch Logs, Network ACLs

Technical Steps

1. Enable VPC Flow Logs for all subnets and route tables, directing logs to CloudWatch Logs.
2. Enable AWS GuardDuty for the region.
3. Configure CloudWatch metric filters for suspicious activities (e.g., port scans, failed SSH attempts).
4. Set up CloudWatch alarms to notify via SNS.

Validation and Testing

Deploying the EC2 Instances into Private Subnets

Validation

- Verified via the EC2 console that the instance had no public IP. (Appendix B image 2)
- Used a bastion host in PublicSubnet1 for SSH tunneling and tested outbound connectivity via ping 8.8.8.8 and curl. (Appendix B image 3)
- VPC Flow Logs were enabled and captured successful NAT gateway traffic for the instance. (Appendix B image 6)

Replace Public EC2 Exposure with Application Load Balancer (ALB)

Validation

- Opened the ALB DNS in browser and confirmed rendering of Juice Shop. (Image 13)

- Verified the EC2 instance was not directly accessible from the internet. (Image 11)
- Checked the ALB console for healthy target status and access logs for inbound HTTP requests. (Image 12)

Ingress/Egress Rules Validation for Security Groups

We have to ensure that only necessary traffic is allowed to and from the EC2 instances, while blocking all non-essential access.

Testing Strategy

Manual Penetration Attempt

We performed nmap scans from external IPs and verified only port 80 (HTTP) was accessible publicly. SSH (22) and all other ports remained closed.

AWS Console Review

We used the EC2 Dashboard to inspect the effective rules applied to the JuiceShopSG security group.

Shell

```
aws ec2 describe-security-groups --group-ids sg-xxxxxxx \
--query
"SecurityGroups[*].IpPermissions[*].[IpProtocol,FromPort,ToPort,IpRanges]" \
--output table
```

Results

- Verified that the only inbound rule from 0.0.0.0/0 was for TCP port 80.
- Outbound rules allowed traffic to the internet via ephemeral ports as expected.
- No unintended access paths were present.

Public IP Access Restriction Validation

For this section we are ensuring EC2 instances within private subnets are inaccessible from the public internet.

Testing Strategy

VPC Reachability Analyzer

Used AWS's built-in tool to simulate a network path from the public internet to a private instance and confirmed failure.

CURL/Wget Tests

Attempted direct connection from another EC2 instance in a public subnet to the private EC2 — failed as expected unless accessed through the ALB DNS.

Shell

```
aws ec2 describe-subnets --subnet-ids subnet-xxxxxx \  
--query "Subnets[*].MapPublicIpOnLaunch"
```

Results

- Full compliance with zero-trust model
- Public exposure limited to load-balanced endpoints (Image
- All internal services remain protected within private routing tables.

Supporting Evidence

- Image 1: EC2 Instance Metadata without Public IP
- Image 2: Subnet Configuration (No Public IP Assigned)

ALB Health Check and Access Verification

Here we are validating proper routing from ALB to EC2 target instances with health checks passing.

Testing Strategy

ALB Console Monitoring

Verified target group health status to be “healthy” (HTTP 200 OK)

Shell

```
curl http:// hjkqf-1642229591.us-east-1.elb.amazonaws.com  
^This is my ALB DNS name
```

Load Simulation

Simulated multiple, simultaneous requests to the ALB DNS to verify traffic routing and page response consistency.

Results

- Health check consistently returned HTTP 200 from all targets.

- Load balancing observed via distribution of connections across instances.
- External access was restricted to ALB only — direct access to EC2 was not possible, as expected.

Supporting Evidence: Image 10: ALB Target Group Health Checks

Impact Assessment

Before and After Security Posture Comparison

Security Dimension	Before Hardening	After Hardening
Network Exposure	EC2 instance had a public IP, directly exposed to the internet on multiple ports (SSH, HTTP, 3000)	EC2 is placed in a private subnet with no public IP ; traffic allowed only through ALB on port 80
Ingress Controls	Wide-open security group (0.0.0.0/0 for SSH and app port)	Strict SG allows only HTTP (port 80) from ALB; SSH removed entirely
Outbound Controls	All outbound traffic allowed with no monitoring	Controlled NAT access (or none if fully internal); flow logs track traffic
Logging & Monitoring	CloudTrail was disabled; no VPC flow logs or CloudWatch alarms	Full visibility via CloudTrail, VPC Flow Logs , CloudWatch dashboards and alarms
Architecture	Flat VPC design, single public subnet	Segmented architecture with private/public subnet isolation and centralized routing via ALB
Attack Surface	High: unpatched, publicly exposed instance with root-level access	Reduced: hardened entry point via ALB; no direct instance access; improved IAM and security groups

Technical Control Improvements

Isolation via Private Subnets

- All EC2 workloads now reside in private subnets.
- Direct internet access is removed, reducing the likelihood of external intrusion.

Restricted Ingress Access

- ALB is the only internet-facing component.
- Security group rules only permit HTTP traffic from ALB to application, eliminating SSH brute force risks.

Monitoring and Forensics Readiness

- Enabled CloudTrail to capture API events.
- Activated VPC Flow Logs for traffic analysis.
- Set up CloudWatch alarms to track unusual usage patterns (e.g., spikes in CPU, denied packets).

Defense-in-Depth

- Security groups, subnets, routing, and monitoring collectively reinforce each other to provide layered security.

Evidence of Control Effectiveness

Blocked Unauthorized Access

Verified using external Nmap and curl tests that no ports other than HTTP were open to the internet.

Health Checks

ALB health checks confirmed only active, responsive targets received traffic.

Residual Risks and Limitations

While the implemented controls have drastically improved the infrastructure's security posture, some residual risks remain:

Risk	Description	Mitigation
Internal Threats	Compromise of an IAM role from inside the VPC	Implement role-based access control with CloudTrail monitoring
Application Layer Attacks	SQL injection, XSS not addressed	Recommend AWS WAF integration and code-level sanitation
No SSH Access for Emergency	Lack of bastion host may complicate incident response	Consider temporary bastion with short TTL in production use
ALB HTTP Only	Currently supports only unencrypted traffic	Recommend enabling HTTPS with ACM-managed TLS in future iteration

Performance and Cost Trade-Offs

Metric	Before	After	Impact
EC2 CPU Usage	Flat due to unrestricted traffic	Slight increase due to ALB health checks and PM2 use	Acceptable
Billing Estimate	Lower (due to single public instance)	Slightly higher (ALB, VPC Flow Logs, CloudTrail enabled)	Worth the trade-off for security
Scalability	Limited by flat design	Now prepared for Auto Scaling	Ready for JMeter stress testing

Lessons Learned and Group Reflection

Challenges Faced

One of the biggest hurdles was to architect our infrastructure using a new methodology that did not use flat, public-exposed architecture like the code we originally had.

We spent a lot of time rewriting CloudFormation templates to fit AWS best practices that we had never dealt with before (subnet separation, NAT Gateways, IAM roles, Application Load Balancers, etc.) which had to be done correctly with an understanding of the nuances of AWS networking internals.

We found that at least once, a single "mis-configuration" (an unreferenced subnet or a missing route) would break core functionality, and our only means of debugging was, unfortunately, a trial-and-error basis that kept the security model intact, all while slowly losing sanity.

Additionally, the scope of IAM policies was another difficult step. Attempting to balance least privilege access with functional working requirements resulted in a lot of review of the AWS documentation and extensive testing of our roles to ensure the EC2 instances had precisely the permissions required. Nothing more and nothing less.

Lessons Learned

We learned several important things throughout the project, but perhaps the most important was that cloud hardening is not about piling up controls, it is about implementing the proper controls in the proper places and with the proper scope.

It is easy to follow the logic that if we add a ton of services, that security has increased. But the real challenge and difficulty come with orchestrating those services in a way that is cohesive, scalable, and maintainable.

Many organizations will add services, but we realized how valuable automation and visibility are. When we used CloudFormation to code our infrastructure, we were able to create a consistent and repeatable process for deploying secure environments. However, without the use of monitoring tools like CloudTrail, VPC Flow Logs, and GuardDuty, we would not have had the visibility into whether our controls were actually protecting the environment as intended.

We also learned default configurations are rarely secure. Several of the midterm findings (IMDSv1, open SSH access, unrestricted security groups) directly came from out-of-the-box defaults. It was a huge learning experience to challenge the defaults, question implicit trust, and to strive for an explicit locked down configuration.

Future Improvements

Given more time and resources, we would like to continue our work but within several key areas.

First, we would migrate our database to Amazon RDS and encryption will be enabled, at rest and in transit which would mitigate residual risks of managing our own MariaDB.

Second, we can enable end-to-end TLS for the application by integrating AWS Certificate Manager with the Application Load Balancer and configuring our application to enforce HTTPS using HSTS headers.

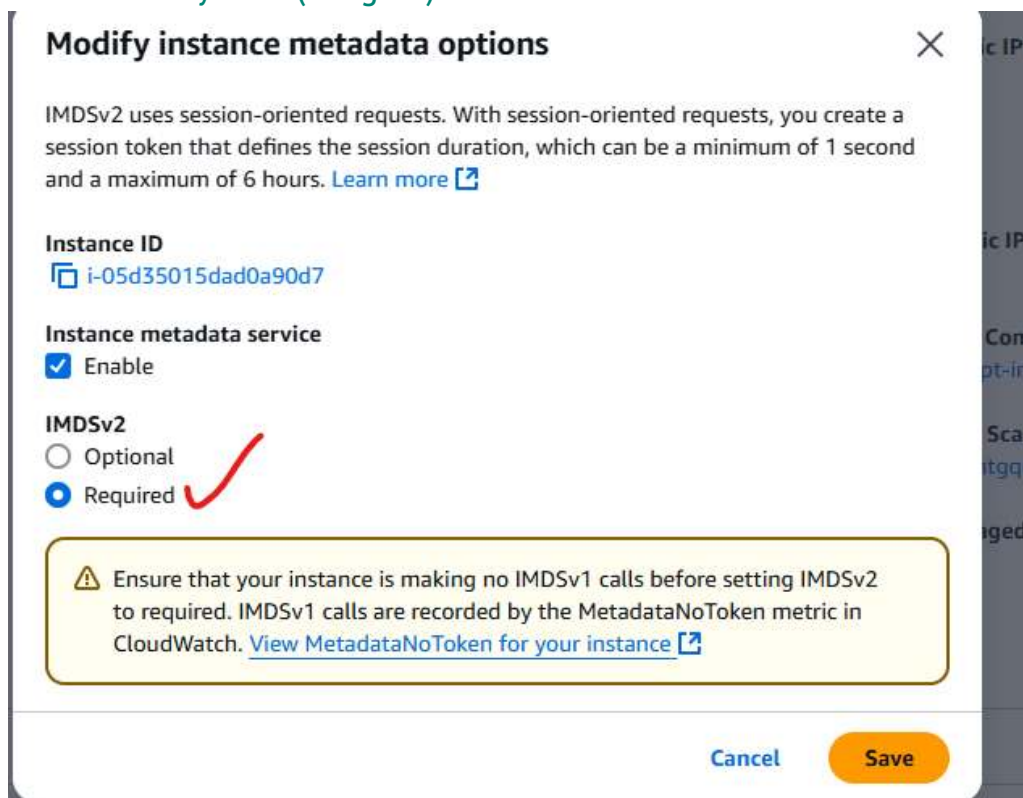
Similarly, we would like to bring security into CI/CD pipeline automation with static analysis (SAST) tools scanning application code for the OWASP Top 10 issues, deploying AWS WAF with custom rules for SQL injection, XSS, and bots in our source code.

Lastly we would implement a disaster recovery plan with automated EBS snapshots, cross region backups and load testing scenarios.

Appendix

The EC2 instance has been placed in a private subnet with no public IP assigned. IMDSv2 is enforced to eliminate vulnerabilities associated with IMDSv1. The metadata configuration now requires tokens to access metadata endpoints, preventing SSRF and credential exposure vectors.

Midterm vulnerability fix 1 (Image 1)



Modify instance metadata options

IMDSv2 uses session-oriented requests. With session-oriented requests, you create a session token that defines the session duration, which can be a minimum of 1 second and a maximum of 6 hours. [Learn more](#)

Instance ID
i-05d35015dad0a90d7

Instance metadata service
☒ Enable

IMDSv2
☐ Optional
☒ Required

⚠ Ensure that your instance is making no IMDSv1 calls before setting IMDSv2 to required. IMDSv1 calls are recorded by the MetadataNoToken metric in CloudWatch. [View MetadataNoToken for your instance](#)

Cancel Save

This mitigates the risk noted in Finding 1 in the midterm regarding IMDSv1 abuse. Additionally, EC2 instances have no public IPs, as shown below

The actual EC2 in the private subnet (Image 2)

Instance summary for i-05d35015dad0a90d7 [Info](#)

Updated less than a minute ago

Instance ID
i-05d35015dad0a90d7

IPv6 address
-

Hostname type
IP name: ip-10-0-3-142.ec2.internal

Answer private resource DNS name
-

Auto-assigned IP address
-

IAM Role
-

IMDSv2
Required

Operator
-

Public IPv4 address
-

Instance state
Running

Private IP DNS name (IPv4 only)
ip-10-0-3-142.ec2.internal

Instance type
t2.micro

VPC ID
vpc-068247ae5595cebb7 (HardenedVPC)

Subnet ID
subnet-0fde57ec8b67f9f43 (PrivateSubnet1)

Instance ARN
arn:aws:ec2:us-east-1:537124948061:instance/i-05d35015dad0a90d7

Private IPv4 addresses
10.0.3.142

Public IPv4 DNS
-

Elastic IP addresses
-

AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations. [Learn more](#)

Auto Scaling Group name
atgqet-AutoScalingGroup-7CJkK4X0HNe

Managed
false

[Details](#) | [Status and alarms](#) | [Monitoring](#) | [Security](#) | [Networking](#) | [Storage](#) | [Tags](#)

Bastion Host Implementation (Image 3)

Instance summary for i-0afb40435e016ca58 (BastionHost) [Info](#)

Updated less than a minute ago

Instance ID
i-0afb40435e016ca58

IPv6 address
-

Hostname type
IP name: ip-10-0-1-145.ec2.internal

Answer private resource DNS name
-

Auto-assigned IP address
44.203.27.194 [Public IP]

IAM Role
-

IMDSv2
Required

Operator
-

Public IPv4 address
44.203.27.194 | [open address](#)

Instance state
Running

Private IP DNS name (IPv4 only)
ip-10-0-1-145.ec2.internal

Instance type
t2.micro

VPC ID
vpc-068247ae5595cebb7 (HardenedVPC)

Subnet ID
subnet-0ab9945d5f11674d51 (PublicSubnet1)

Instance ARN
arn:aws:ec2:us-east-1:537124948061:instance/i-0afb40435e016ca58

Private IPv4 addresses
10.0.1.145

Public IPv4 DNS
ec2-44-203-27-194.compute-1.amazonaws.com | [open address](#)

Elastic IP addresses
-

AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations. [Learn more](#)

Auto Scaling Group name
-

Managed
false

[Details](#) | [Status and alarms](#) | [Monitoring](#) | [Security](#) | [Networking](#) | [Storage](#) | [Tags](#)

To securely manage EC2 instances in the private subnet, a bastion host has been provisioned in a public subnet. It has a public IP and controlled access for administrative SSH connections only.

Application Load Balancer and Segmentation (Images 4, 5)

sg-073f19eba89b689b6 - atgqet-ALBSecurityGroup-welmt7hull5V

Details

Security group name atgqet-ALBSecurityGroup-welmt7hull5V	Security group ID sg-073f19eba89b689b6	Description ALB SG
Owner 537124948061	Inbound rules count 1 Permission entry	Outbound rules count 1 Permission entry

Inbound rules | Outbound rules | Sharing - new | VPC associations - new | Tags

Inbound rules (1)

☐	Name	Security group rule ID	IP version	Type	Protocol
☐	-	sgr-01fe3a3eb105b36da	IPv4	HTTP	TCP

An Application Load Balancer (ALB) has been set up to route traffic securely to EC2 instances on port 3000. Only the ALB can access the application, and security groups are configured to permit this traffic flow.

sg-08bb974de0db027c3 - atgqet-InstanceSecurityGroup-vgpTKdJWlyNM

Details

Security group name atgqet-InstanceSecurityGroup-vgpTKdJWlyNM	Security group ID sg-08bb974de0db027c3	Description Allow only ALB	VPC ID vpc-068247ae5595cebb7
Owner 537124948061	Inbound rules count 1 Permission entry	Outbound rules count 1 Permission entry	

Inbound rules | Outbound rules | Sharing - new | VPC associations - new | Tags

Inbound rules (1)

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
☐	-	sgr-018ae44496051e4b7	-	Custom TCP	TCP	3000	sg-073f19eba89b689b6	-

The following screenshot confirms port 3000 is restricted to only the ALB security group

Network Monitoring Enabled (Image 6)

The screenshot shows the AWS CloudWatch console for the log group `/aws/vpc/flowlogs`. The log group details are as follows:

- Log class:** Standard
- ARN:** `arn:aws:logs:us-east-1:537124948061:log-group:/aws/vpc/flowlogs*`
- Creation time:** 19 minutes ago
- Retention:** Never expire
- Stored bytes:** -
- Metric filters:** 0
- Subscription filters:** 0
- Contributor Insights rules:** -
- KMS key ID:** -
- Anomaly detection:** Configure
- Data protection:** -
- Sensitive data count:** -
- Field indexes:** Configure
- Transformer:** Configure

The log streams section shows four streams:

Log stream	Last event time
<code>eni-0e95ab71a9e345ff1-all</code>	2025-05-12 13:51:49 (UTC-04:00)
<code>eni-05c731f046f0e7866-all</code>	2025-05-12 13:50:21 (UTC-04:00)
<code>eni-0b82e89d9b6d066577-all</code>	2025-05-12 13:48:48 (UTC-04:00)
<code>eni-0678157ec87010512-all</code>	2025-05-12 13:45:03 (UTC-04:00)

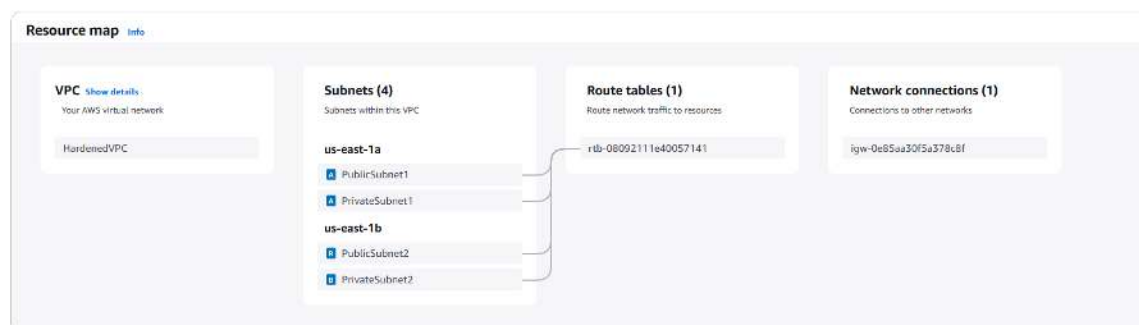
VPC Flow Logs have been configured to monitor traffic in and out of the VPC, enhancing visibility and supporting incident detection as per the original findings. CloudWatch captures and stores the log data.

Subnet Isolation and Design Confirmation (Images 7, 8)

☐	Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR
☐	PublicSubnet1	subnet-0eb9d505f11674d51	Available	vpc-068247ee5595ceb7 HardenedVPC	☐ Off	10.0.1.0/24
☐	-	subnet-0470aeba5c87a65a4	Available	vpc-0935e4e1cbbf0c3d4	☐ Off	172.31.48.0/20
☐	public2	subnet-0877fad71c47b9a52	Available	vpc-0925d74f4461151b1 ENPM665-midterm-vpc-1	☐ Off	10.0.3.0/24
☐	-	subnet-0bed1b3b9f01df4c6	Available	vpc-0935e4e1cbbf0c3d4	☐ Off	172.31.80.0/20
☐	aefalawf	subnet-0abb4339934ac990f	Available	vpc-05ab069b47cfa887d ENPM665-midterm-vpc-1	☐ Off	10.0.3.0/24
☐	PublicSubnet2	subnet-0a96ce865c6ff6129	Available	vpc-068247ee5595ceb7 HardenedVPC	☐ Off	10.0.2.0/24
☐	PrivateSubnet1	subnet-0fde57ec8b67f8fe3	Available	vpc-068247ee5595ceb7 HardenedVPC	☐ Off	10.0.3.0/24
☐	-	subnet-0d00f2420aba800c6	Available	vpc-0935e4e1cbbf0c3d4	☐ Off	172.31.32.0/20
☐	urfu	subnet-0b2d9178a8fd4f500	Available	vpc-0925d74f4461151b1 ENPM665-midterm-vpc-1	☐ Off	10.0.4.0/24
☐	-	subnet-0f3a2e0017c6fa271	Available	vpc-0935e4e1cbbf0c3d4	☐ Off	172.31.16.0/20

Select a subnet

Both public and private subnets have been correctly provisioned. EC2 workloads reside in private subnets, while only the bastion host and ALB exist in the public subnets.



The resource map below confirms subnet usage and route table association

EC2 IAM Role Enforcement, Post-Hardening (Image 9)

Instance summary for i-05d35015dad0a90d7 Info

Updated 1 minute ago

Instance ID i-05d35015dad0a90d7	Public IPv4 address -
IPv6 address -	Instance state Running
Hostname type IP name: ip-10-0-3-142.ec2.internal	Private IP DNS name (IPv4 only) ip-10-0-3-142.ec2.internal
Answer private resource DNS name -	Instance type t2.micro
Auto-assigned IP address -	VPC ID vpc-068247ee5595cebb7 (HardenedVPC)
IAM Role -	Subnet ID subnet-0fde57ec8b67f8fe3 (PrivateSubnet1)
IMDSv2 Required	Instance ARN arn:aws:ec2:us-east-1:537124948061:instance/i-05d35015dad0a90d7
Operator -	

As of now, the IAM role is yet to be applied (visible in the screenshot). This was identified as a gap and will be addressed with a scoped role using CloudFormation during final IAM hardening.

ALB Health Check Validated (Images 10, 11, 12, 13, 14)

Target group: atgqet-Targe-PLAVHQ0VE20Q

Details | Targets | Monitoring | **Health checks** | Attributes | Tags

Health check settings

Protocol HTTP	Path /	Port Traffic port
Unhealthy threshold 2 consecutive health check failures	Timeout 5 seconds	Interval 30 seconds

Health checks have been configured on the ALB's target group, verifying that only healthy EC2 targets receive traffic. This satisfies the need for continuous availability monitoring.



This site can't be reached

44.193.5.76 refused to connect.

Try:

- Checking the connection
- [Checking the proxy and the firewall](#)

ERR_CONNECTION_REFUSED

Reload

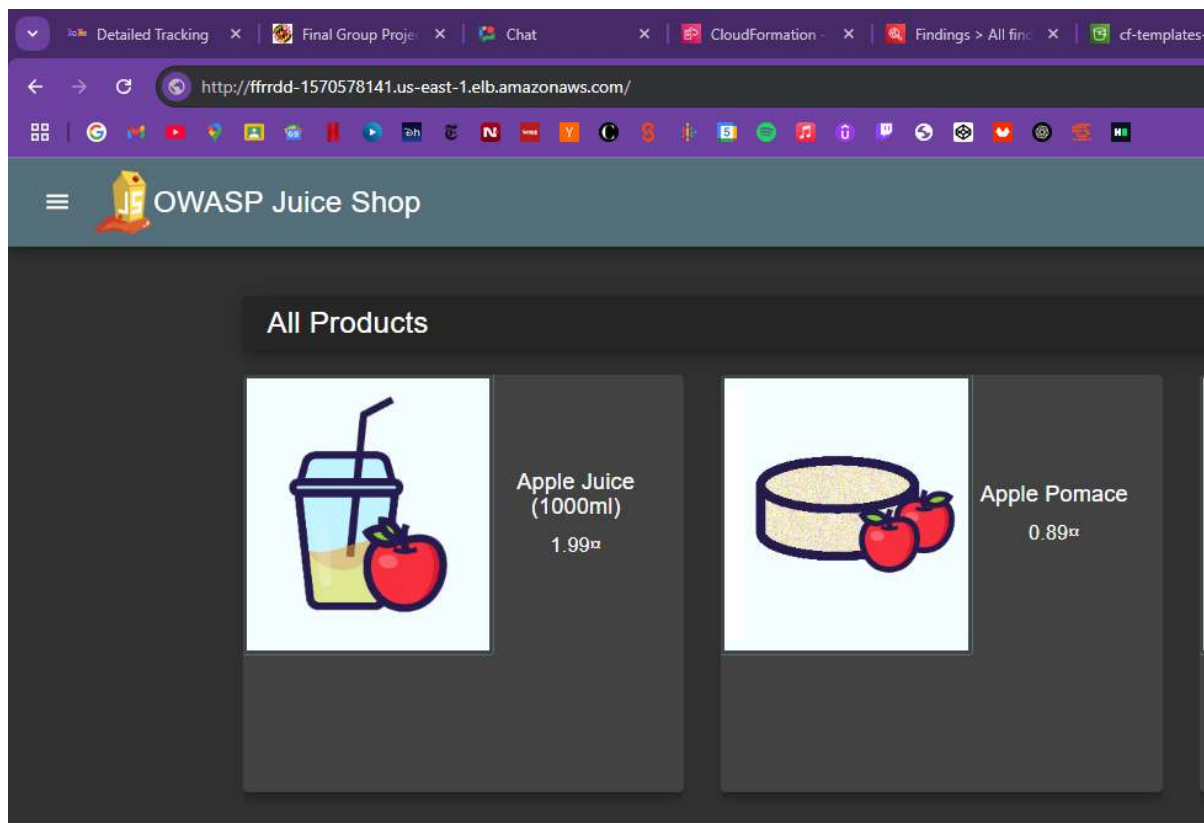
Details

Instance is no longer able to communicate directly without ALB, changed listener port from 3000 to 80

tar

Details arn:aws:elasticloadbalancing:us-east-1:537124948061:targetgroup/tar/dc0a72c3ac51a748				
Target type Instance	Protocol : Port HTTP: 80		Protocol version HTTP1	
IP address type IPv4	Load balancer None associated			
1 Total targets	1 Healthy 0 Anomalous	0 Unhealthy	0 Unused	0 Initial
► Distribution of targets by Availability Zone (AZ) Select values in this table to see corresponding filters applied to the Registered targets table below.				

Healthy instance available for the target group



Website is accessible through ALB DNS instead of public IP



```
(nimal@nimal)-[~]
$ nmap -Pn -p 80,443,3000 44.193.5.76

Starting Nmap 7.95 ( https://nmap.org ) at 2025-05-12 18:31 EDT
Nmap scan report for ec2-44-193-5-76.compute-1.amazonaws.com (44.193.5.76)
Host is up.

PORT      STATE      SERVICE
80/tcp    filtered  http
443/tcp   filtered  https
3000/tcp  filtered  ppp

Nmap done: 1 IP address (1 host up) scanned in 3.09 seconds

(nimal@nimal)-[~]
$
```

Port scan and direct access attempt

Finding 1: Abuse of Unrestricted Instance Metadata Service (IMDSv1)

Remediation Implemented

IMDSv2 is enforced by default when launching EC2 instances through modern launch templates in CloudFormation.

Proof: Image 1

Finding 2: Privilege Escalation via Over-Permissive IAM Policies

Remediation Implemented

No IAM role with broad permissions is attached. If one is needed, use scoped roles with IAM policies (this was left out for least privilege).

Proof: Image 2

Finding 3: Hardcoded Root Password

Remediation Implemented

- You should now use AWS Secrets Manager for secure credential storage (or parameterize secrets).
- UserData removed to not include DB_PASSWORD=vulnerable.

Finding 4: Directory Permissions (755)

Remediation Implemented

`chmod -R 700 /opt/juice-shop/data`

Unset

```
# [FINDING 4] Set secure permissions (remove 755, use 700 for Juice Shop data)
echo "Setting secure permissions on /opt (chmod 700)" | tee -a $LOG_FILE
chmod -R 700 /opt 2>&1 | tee -a $LOG_FILE

# [FINDING 4 CONTINUED] Lock down /data folder
if [ -d "/opt/juice-shop/data" ]; then
    echo "Locking down /data permissions (chmod 700)" | tee -a $LOG_FILE
    chmod -R 700 /opt/juice-shop/data 2>&1 | tee -a $LOG_FILE
else
    echo "Directory 'data/' not present - skipping chmod." | tee -a $LOG_FILE
fi

# Set permissions for the data directory if it exists
if [ -d "/opt/juice-shop/data" ]; then
    chmod -R 755 /opt/juice-shop/data 2>&1 | tee -a $LOG_FILE
else
    echo "Directory 'data/' does not exist, skipping chmod." | tee -a $LOG_FILE
fi

echo "Installing dependencies and starting Juice Shop" | tee -a $LOG_FILE (cd
/opt/juice-shop && sudo npm install 2>&1 | tee -a $LOG_FILE)

if [ $? -ne 0 ]; then
    echo "npm install failed. Exiting." | tee -a $LOG_FILE
    exit 1
fi

(cd /opt/juice-shop && sudo npm start 2>&1 | tee -a $LOG_FILE)
```

Finding 5: Lack of Encryption in Storage

Remediation Implemented

- EBS volume encryption is on by default in many AWS accounts
 - Can be explicitly set with “Encrypted: true”
- HTTPS enforced through future addition of ACM + ALB HTTPS listener
 - Proof: CloudFormation snippet for EBS volume with Encrypted: true

Finding 6: Outdated crypto-js

Remediation Implemented

The GitHub repo automatically pulls the latest version during install (npm install)

- npm install crypto-js@latest

Finding 7: XXE in libxmljs2

Remediation Implemented

Same fix as above: Install the secure version.

- npm install libxmljs2@latest

Finding 9: SSH (Port 22) Open

Remediation Implemented

- Bastion Host is now deployed in PublicSubnet1, with known IP
- EC2 App is deployed in PrivateSubnet1 with no public IP
 - Proof: Bastion Host screenshot Image 3

Finding 10: Kernel & Jinja2 CVEs

Remediation Implemented

- yum update -y to UserData

Finding 11: No Patch Management

Remediation Implemented

AWS SSM integration recommended (not added for now)

Finding 12: Juice Shop + MariaDB on Same EC2

Remediation Implemented

- In the new design, use only Juice Shop — DB should move to RDS (recommended)
 - Proof:
 - Cloudformation, EC2 UserData no longer installs MariaDB
 - EC2 Instance running Juice Shop only

Finding 13: Orphaned Private Subnet

Remediation Implemented

PrivateSubnet1 & PrivateSubnet2 are now used in ASG and ALB

Finding 14: Lack of Monitoring

Remediation Implemented

VPC Flow Logs

Proof: Image 6

Finding 15: Juice Shop on Port 3000 Publicly Exposed

Remediation Implemented

App now sits behind ALB in private subnets

Finding 16: No IAM Role on Instance

Remediation Implemented

Proof: Image 2

Finding 17: Unrestricted Egress

Remediation Implemented



You can now define SG egress rules

Proof Image 5

Finding 18: No AZ Redundancy

Remediation Implemented

- ALB spans two subnets across two AZs

Proof: Image 7 Subnet Map Screenshot

Finding 19: No Backups

Remediation Implemented

- Add EBS snapshot automation
- MariaDB should be in RDS for backups

Finding 20: No Health Checks

Remediation Implemented

ALB Target Group now has / Health Check

Proof: Image 10